



2026-1 텍스트마이닝

과제_1 텍스트 크롤링 및 텍스트 전처리

세종대학교
소프트웨어학과
학번: 22011831
이름: 김형규



Step 0-2: HTML 구조 탐색 (크롤링 전 분석)

```
import requests
from bs4 import BeautifulSoup

url = "https://en.wikipedia.org/wiki/Machine_learning"
headers = {"User-Agent": "Mozilla/5.0"}

res = requests.get(url, headers=headers)
soup = BeautifulSoup(res.text, "html.parser")

all_p = soup.find_all("p") # 🍌 여기를 채우세요!
print(f"전체 <p> 태그 개수: {len(all_p)}개")

first_p = soup.find("p") # 🍌 여기를 채우세요!
print(first_p.get_text()[:300])

content_p = soup.select("#mw-content-text") # 🍌 여기를 채우세요!
print(f"본문 <p> 태그 개수: {len(content_p)}개")
print(f"\n첫 번째 문단:\n{content_p[0].get_text()[:300]}")
```

전체 <p> 태그 개수: 114개

본문 <p> 태그 개수: 1개

첫 번째 문단:

Study of algorithms that improve automatically through experience
For the journal, see Machine Learning (journal).
"Statistical learning" redirects here. For statistical learning in linguistics, see Statistical learning

Q1. 본문 문단은 어떤 태그로 감싸져 있나요?

A1. <p> 태그

Q2. 본문 영역을 감싸는 상위 <div>의 id는 무엇인가요?

A2. #mw-content-text

Q3. 본문만 정확히 가져오려면 어떤 CSS 선택자를 써야 할까요?

A3. #mw-content-text p

위키피디아 사이트에서 개발자모드를 통해 HTML 태그 분석을 해보면,
"mw-content-text"라는 id를 가진 태그가 본문을 감싸고 있는 것을 알 수 있다.

#mw-content-text p를 통해 본문의 텍스트만 정확히 가져올 수 있다.



Step 1-1: Wikipedia 랜덤 제목 100개 수집

```
import requests
import pandas as pd
from bs4 import BeautifulSoup

headers = {
    "User-Agent": "Mozilla/5.0"
}

url = "https://en.wikipedia.org/w/api.php"

# API 파라미터 설정
# 참고: https://www.mediawiki.org/wiki/API:Random
params = {
    "action": "query",
    "format": "json",
    "list": "random",
    "rnnamespace": 0,
    "rnlimit": 100 # 🍌 여기를 채우세요!
}

response = requests.get(url, params=params, headers=headers)

data = response.json()

print(data)
```

```
{'batchcomplete': '', 'continue': {'rncontinue': '0.478402226583|0.478416660512|1129884|0', 'continue': '-||'}, 'query': {'random': [{'id': 54611803, 'ns': 0, 'title': 'SEK Class Theta-beta'}, {'id': 65716698, 'ns': 0, 'title': 'Kengo Yamazaki'}, {'id': 33566090, 'ns': 0, 'title': 'Winye language'}, {'id': 19035693, 'ns': 0, 'title': 'Jankowce'}, {'id': 37156884, 'ns': 0, 'title': '2012 ARIA Music Awards'}, {'id': 7676976, 'ns': 0, 'title': 'Russett, Maryland'}, {'id': 41148554, 'ns': 0, 'title': 'Wilson Mukama'}, {'id': 23058658, 'ns': 0, 'title': 'Ollie King'}, {'id': 76116295, 'ns': 0, 'title': 'Bichhiya, Madhya Pradesh'}, {'id': 57202800, 'ns': 0, 'title': 'Experimental Psychology (journal)'}, {'id': 61837215, 'ns': 0, 'title': 'Greg Gbur'}, {'id': 1269279, 'ns': 0, 'title': 'Earl Campbell'}, {'id': 11049618, 'ns': 0, 'title': 'David Mack (police officer)'}, {'id': 3252212, 'ns': 0, 'title': 'Voluntary dismissal'}, {'id': 998116, 'ns': 0, 'title': 'Node (networking)'}, {'id': 1820880, 'ns': 0, 'title': 'Skipton and Ripon'}, {'id': 25615230, 'ns': 0, 'title': 'Henry Gale (cricketer)'}, {'id': 2224369, 'ns': 0, 'title': 'Aspern'}, {'id': 19829874, 'ns': 0, 'title': 'Zwola, Garwolin County'}, {'id': 104
```

위키피디아 API와 BeautifulSoup 라이브러리를 이용해 랜덤 문서 100개의 제목을 가져온다.

Params 파라미터 설정을 통해 응답 형식(JSON), 가져올 문서 개수(rnlimit)등을 정의한다.



Step 1-1: Wikipedia 랜덤 제목 100개 수집

```
# 위에서 출력한 JSON 구조를 보고, title만 추출하세요
titles = [item["title"] for item in data["query"]["random"]] # 🙋 여기를 채우세요!

print("수집된 문서 수:", len(titles))
print("예시 제목:", titles[:5])
```

수집된 문서 수: 100

예시 제목: ['SEK Class Theta-beta', 'Kengo Yamazaki', 'Winye language', 'Jankowce', '2012 ARIA Music Awards']

앞에서 가져온 JSON 형식 구조에서 title만 추출하여 titles 리스트에 저장한다.



Step 1-2: 각 Wikipedia 페이지 본문(content) 크롤링

```
texts = []
refs = []

for title in titles:
    page_url = f"https://en.wikipedia.org/wiki/{title.replace(' ', '_')}}"
    res = requests.get(page_url, headers=headers)
    soup = BeautifulSoup(res.text, "html.parser")

    # References 섹션 따로 추출 (decompose 전에!)
    ref_tags = soup.select(".reflist, .references")
    ref_text = " ".join([r.get_text() for r in ref_tags])

    # References 섹션 제거 후 본문 추출
    for tag in ref_tags:
        tag.decompose()

    paragraphs = soup.select("#mw-content-text p") # 👉 여기를 채우세요!
    main_text = " ".join([p.get_text() for p in paragraphs])

    texts.append(main_text)
    refs.append(ref_text)
```

앞에서 저장한 title리스트를 이용하여 위키피디아 크롤링을 진행하고, 본문 내용과 references 섹션의 내용들을 따로 저장해준다.

-> 본문의 내용으로만 전처리를 진행하면, 'retrieved', 'original', 'archived' 등과 같은 references 섹션에서 등장하는 단어들은 대부분의 문서에 있기 때문에, Frequency 계산 시 노이즈를 줄일 수 있다.



Step 1-3: 데이터프레임 생성

Step 1-4: 데이터 확인

```
df = pd.DataFrame({
    "title": titles,
    "content": texts,
    "references": refs
})

df.head()
```

	title	content	references
0	SEK Class Theta-beta	Wn SEK (Sidirodromoi Ellinikou Kratous, Hellen...	Wn^ Durrant 1972, p. 52.WnWn
1	Kengo Yamazaki	Wn Kengo Yamazaki (山崎 謙吾, Yamazaki Kengo; born...	Wn^ "Profile" (in Japanese). TBS. Retrieved 30...
2	Winye language	Winyé, or Kolsi, is a Gur language of Burkina ...	Wn^ Winyé at Ethnologue (18th ed., 2015) (subs...
3	Jankowce	Jankowce [jan'kɔftɕɛ] is a village in the adm...	Wn^ "Główny Urząd Statystyczny" [Central Stati...
4	2012 ARIA Music Awards	Wn The 26th Annual Australian Recording Indust...	Wn^ Public voted category "Single of the Year"...

크롤링한 데이터를 데이터프레임으로 정리해준다.

```
print("총 문서 수:", len(df))
print("첫 번째 문서 길이:", len(df["content"][0]))
```

총 문서 수: 100
첫 번째 문서 길이: 743



Step 2: 텍스트 전처리(Cleaning) 목표

```
import re

def clean_text(text):
    # 소문자 변환
    text = text.lower() # 🙌 여기를 채우세요!

    # [edit], [update] 같은 위키 태그 제거 ← 추가
    text = re.sub(r'\[edit\]', ' ', text)

    # [1], [2] 같은 citation 제거
    text = re.sub(r'\[\d+\]', ' ', text) # 🙌 여기를 채우세요!

    # 특수문자/숫자 제거
    text = re.sub(r'^a-z\s', ' ', text) # 🙌 여기를 채우세요!

    return text

df["clean_text"] = df["content"].apply(clean_text)
df[["content", "clean_text"]].head(3)
```

	content	clean_text
0	Wn SEK (Sidirodromoi Ellinikou Kratous, Hellen...	Wn sek sidirodromoi ellinikou kratous hellen...
1	Wn Kengo Yamazaki (山崎 謙吾, Yamazaki Kengo; born...	Wn kengo yamazaki yamazaki kengo born...
2	Winyé, or Kolsi, is a Gur language of Burkina ...	winy or kolsi is a gur language of burkina ...

가져온 corpus 데이터를 clean_text로 변환해주는 함수이다.

- 소문자로 변환
- [edit], [update]와 같은 위키 태그 제거
- [1], [2]와 같은 citation 제거
- 특수문자 및 숫자 제거

데이터프레임에 본문을 clean_text로 변환한 텍스트들을 저장한다.



Step 3: Tokenization + Stopwords 제거

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# 영어 불용어 리스트 로드
stop_words = set(stopwords.words("english"))

def tokenize(text):
    # 토큰화
    tokens = word_tokenize(text) # 👉 여기를 채우세요!
    # 불용어 제거 + 길이 2 이하 제거
    tokens = [w for w in tokens if w not in stop_words and len(w) > 2] # 👉 여기를 채우세요!

    return tokens

df["tokens"] = df["clean_text"].apply(tokenize)
df["tokens"].head()
```

```
0    [sek, sidirodromoi, ellinikou, kratous, hellen...
1    [kengo, yamazaki, yamazaki, kengo, born, july,...
2    [winy, kolsi, gur, language, burkina, faso, sp...
3    [jankowce, village, administrative, district, ...
4    [annual, australian, recording, industry, asso...
Name: tokens, dtype: object
```

Clean_text로 정제된 텍스트를 단어(Token)단위로 쪼개고 불용어를 제거하는 코드이다.

Nltk라이브러리에서 불용어 리스트를 가져온 뒤, 불용어와 길이가 2 이하인 단어들을 제거한다.



Step 4: Word Frequency 분석

```
from collections import Counter
# 모든 토큰을 하나로 합치기
all_words = sum(df["tokens"], [])
all_words
```

```
['sek', ...]
```

```
# Counter 사용하여 빈도 계산하기
word_freq = Counter(all_words)
# 상위 20개 단어 출력
word_freq.most_common(20)
```

```
[('first', 149),
 ('also', 118),
 ('one', 99),
 ('campbell', 95),
 ('team', 64),
 ('year', 63),
 ('series', 60),
 ('england', 60),
 ('school', 59),
 ('world', 56),
 ('years', 56),
 ('new', 55),
 ('time', 53),
 ('two', 52),
 ('football', 52),
 ('sri', 52),
 ('university', 51),
 ('...', 51)]
```

모든 문서의 토큰을 합친 뒤, 각 토큰의 등장 빈도를 계산하는 코드이다.

[illegible]

first, one, year와 같은 단어들이 자주 등장한 것을 알 수 있다.



```
os.path.exists("wikipedia_text_dataset.csv")
```

True

데이터프레임을 나중에 학습을 위해 .csv파일로 저장한다.

[illegible]